

PROYECTO MÓDULO #5

ALKE WALLET

Fundamentos de Bases de Datos Relacionales

Estudiante: Alba Moreno

Proyecto ABP - Diseño e implementación de base de datos relacional

Septiembre 2026

1. Introducción y objetivo

El proyecto Alke Wallet consiste en diseñar e implementar una base de datos relacional para una billetera virtual. La base permite almacenar información de usuarios, monedas y transacciones, manteniendo relaciones entre las tablas mediante claves primarias y claves foráneas.

El objetivo del trabajo es aplicar los contenidos del módulo: bases de datos relacionales, consultas SQL, manipulación de datos con DML, definición de estructuras con DDL, transaccionalidad y modelo entidad-relación.

1.1 Requerimientos principales

- Crear una base de datos llamada AlkeWallet.
- Crear las entidades usuario, moneda y transaccion.
- Definir claves primarias, claves foráneas y restricciones NOT NULL.
- Insertar datos de prueba y realizar consultas SELECT.
- Consultar la moneda de un usuario y las transacciones registradas.
- Modificar el correo de un usuario y eliminar una transacción.
- Relacionar tablas mediante JOIN y aplicar funciones de agregación.
- Demostrar el uso de una transacción controlada y ROLLBACK.

1.2 Herramienta utilizada

La implementación y las pruebas se realizaron en MySQL Workbench. Las capturas incluidas en este documento corresponden a las ejecuciones realizadas durante el desarrollo.

2. Creación de la base de datos

Primero se creó la base de datos AlkeWallet y luego se seleccionó para trabajar sobre ella. Esta etapa corresponde a sentencias DDL, ya que define la estructura inicial del proyecto.

```
CREATE DATABASE IF NOT EXISTS AlkeWallet;  
USE AlkeWallet;
```

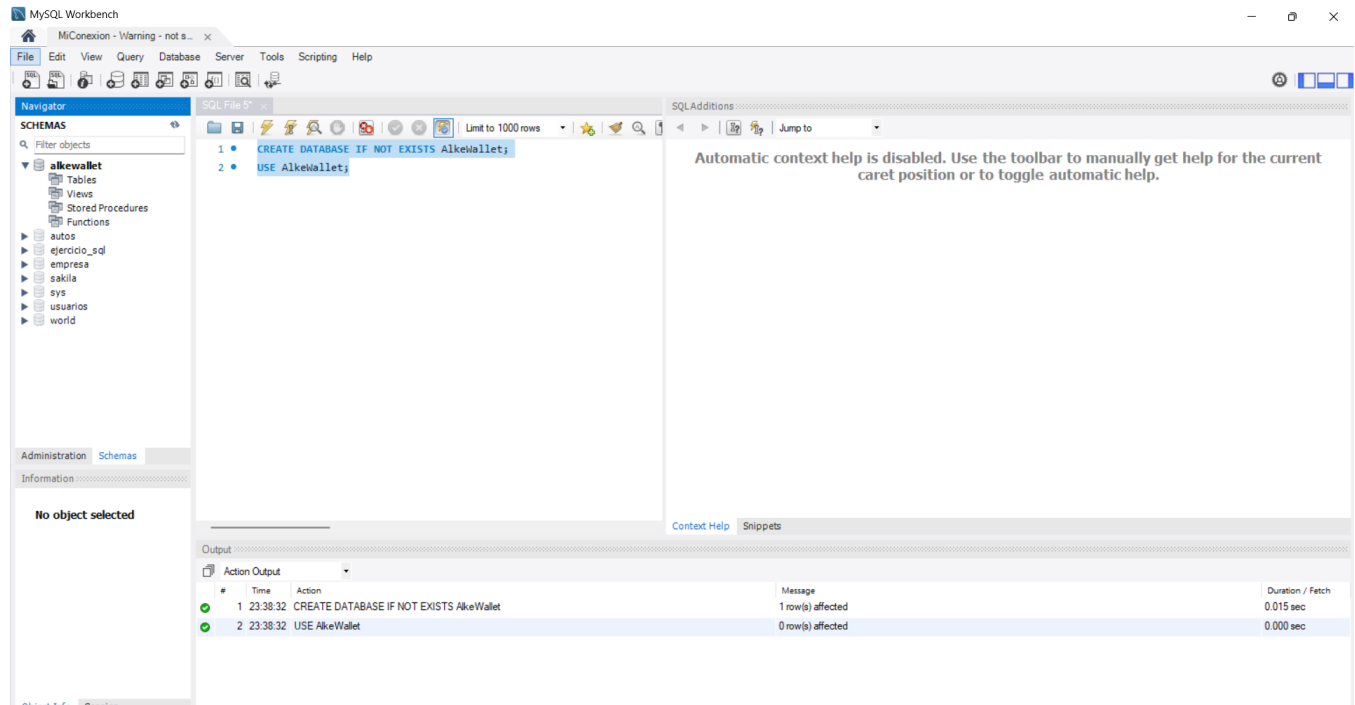


Figura 1. Creación y selección de la base de datos AlkeWallet.

3. Diseño y creación de las tablas

Se crearon tres tablas principales. La tabla moneda almacena las monedas disponibles; usuario almacena los datos y saldo de cada usuario; y transaccion registra los movimientos entre usuarios.

3.1 Tabla moneda

```
CREATE TABLE moneda (  
  currency_id INT AUTO_INCREMENT PRIMARY KEY,  
  currency_name VARCHAR(50) NOT NULL,  
  currency_symbol VARCHAR(10) NOT NULL  
);
```

3.2 Tabla usuario

```
CREATE TABLE usuario (  
  user_id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  correo VARCHAR(100) NOT NULL UNIQUE,  
  contrasena VARCHAR(100) NOT NULL,  
  saldo DECIMAL(12,2) NOT NULL DEFAULT 0,  
  currency_id INT NOT NULL,  
  FOREIGN KEY (currency_id) REFERENCES moneda(currency_id)  
);
```

3.3 Tabla transaccion

```
CREATE TABLE transaccion (  
  transaction_id INT AUTO_INCREMENT PRIMARY KEY,  
  sender_user_id INT NOT NULL,  
  receiver_user_id INT NOT NULL,  
  importe DECIMAL(12,2) NOT NULL,  
  transaction_date DATETIME NOT NULL,  
  FOREIGN KEY (sender_user_id) REFERENCES usuario(user_id),  
  FOREIGN KEY (receiver_user_id) REFERENCES usuario(user_id)  
);
```

The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'Schemas' tree with 'alkewallet' selected, containing tables 'moneda', 'transaccion', 'usuario', 'autos', 'ejercicio_sd', 'empresa', 'sakila', 'sys', 'usuarios', and 'world'. The 'moneda' table is highlighted, showing its columns: 'currency_id' (int, AI, PK), 'currency_name' (varchar(50)), and 'currency_symbol' (varchar(10)).

The main editor shows the SQL script for creating the tables:

```

11 CREATE TABLE usuario (
12   user_id INT AUTO_INCREMENT PRIMARY KEY,
13   nombre VARCHAR(100) NOT NULL,
14   correo VARCHAR(100) NOT NULL UNIQUE,
15   contrasena VARCHAR(100) NOT NULL,
16   saldo DECIMAL(12,2) NOT NULL DEFAULT 0,
17   currency_id INT NOT NULL,
18   FOREIGN KEY (currency_id) REFERENCES moneda(currency_id)
19 );
20
21 -- Tabla transaccion
22 CREATE TABLE transaccion (
23   transaction_id INT AUTO_INCREMENT PRIMARY KEY,
24   sender_user_id INT NOT NULL,
25   receiver_user_id INT NOT NULL,
26   importe DECIMAL(12,2) NOT NULL,
27   transaction_date DATETIME NOT NULL,
28   FOREIGN KEY (sender_user_id) REFERENCES usuario(user_id),
29   FOREIGN KEY (receiver_user_id) REFERENCES usuario(user_id)
30 );

```

The right sidebar shows a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

The bottom output window shows the execution results:

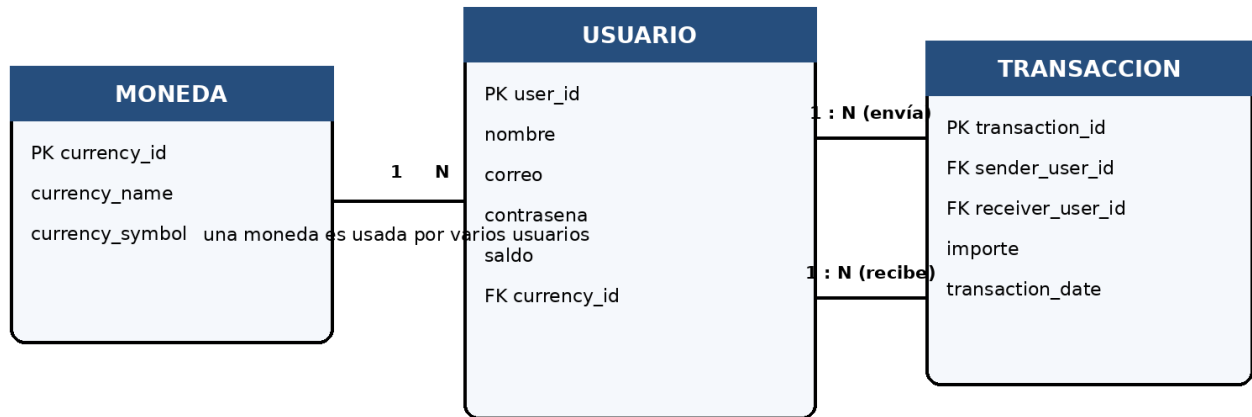
#	Time	Action	Message	Duration / Fetch
1	23:38:32	CREATE DATABASE IF NOT EXISTS AlkeWallet	1 row(s) affected	0.015 sec
2	23:38:32	USE AlkeWallet	0 row(s) affected	0.000 sec
3	23:48:56	CREATE TABLE moneda (currency_id INT AUTO_INCREMENT PRIMARY KEY, currency_name VARCHAR...	0 row(s) affected	0.031 sec
4	23:48:56	CREATE TABLE usuario (user_id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT ...	0 row(s) affected	0.031 sec
5	23:48:56	CREATE TABLE transaccion (transaction_id INT AUTO_INCREMENT PRIMARY KEY, sender_user_id INT...	0 row(s) affected	0.031 sec

Figura 2. Creación de las tablas moneda, usuario y transaccion con claves y restricciones.

4. Modelo entidad-relación

El modelo se organiza en tres entidades fuertes: MONEDA, USUARIO y TRANSACCION. Una moneda puede estar asociada a varios usuarios. A su vez, un usuario puede participar en muchas transacciones como emisor y también en muchas como receptor.

Modelo Entidad-Relación - Alke Wallet



Cardinalidades: MONEDA 1:N USUARIO | USUARIO 1:N TRANSACCION como emisor y como receptor

Figura 3. Diagrama entidad-relación propuesto para Alke Wallet.

4.1 Correspondencia ER → Relacional

Entidad	Clave primaria	Claves foráneas	Relación principal
moneda	currency_id	-	1:N con usuario
usuario	user_id	currency_id	N:1 con moneda; 1:N con transaccion
transaccion	transaction_id	sender_user_id, receiver_user_id	N:1 con usuario como emisor/receptor

4.2 Normalización

El esquema separa la información en tablas según su función. Los datos de moneda no se repiten dentro de cada usuario, sino que se relacionan mediante currency_id. Las transacciones tampoco duplican los datos personales de los usuarios, sino que utilizan sender_user_id y receiver_user_id. De esta forma se reducen redundancias y el diseño se mantiene organizado hasta una estructura compatible con 3FN para los atributos utilizados en el proyecto.

5. Inserción de datos de prueba

Después de crear las tablas se ingresaron datos de prueba mediante INSERT. Se utilizaron tres monedas, tres usuarios y tres transacciones para poder comprobar las consultas solicitadas.

```
INSERT INTO moneda (currency_name, currency_symbol) VALUES
('Peso Chileno', 'CLP'),
('Dolar Estadounidense', 'USD'),
('Euro', 'EUR');
```

```
INSERT INTO usuario
(nombre, correo, contrasena, saldo, currency_id) VALUES
('Ana Perez', 'ana@email.com', 'clave123', 350000, 1),
('Luis Soto', 'luis@email.com', 'clave456', 180000, 1),
('Maria Rojas', 'maria@email.com', 'clave789', 950, 2);
```

```
INSERT INTO transaccion
(sender_user_id, receiver_user_id, importe, transaction_date) VALUES
(1, 2, 25000, '2026-09-10 10:30:00'),
(2, 1, 15000, '2026-09-11 12:15:00'),
(1, 3, 30000, '2026-09-12 17:45:00');
```

Para verificar los registros se ejecutaron SELECT sobre las tres tablas:

```
SELECT * FROM moneda;
SELECT * FROM usuario;
SELECT * FROM transaccion;
```

Durante las pruebas iniciales se repitieron inserciones. Para volver a un estado limpio se desactivaron temporalmente las verificaciones de claves foráneas, se vaciaron las tablas con TRUNCATE y luego se cargaron nuevamente los tres registros de cada tabla.

```
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE transaccion;
TRUNCATE TABLE usuario;
TRUNCATE TABLE moneda;
SET FOREIGN_KEY_CHECKS = 1;
```

6. Consulta 1: moneda elegida por un usuario

Esta consulta relaciona usuario con moneda mediante INNER JOIN. Se utiliza el user_id 1 para obtener la moneda asociada a Ana Perez.

```
SELECT
    u.nombre,
    m.currency_name,
    m.currency_symbol
FROM usuario u
INNER JOIN moneda m
    ON u.currency_id = m.currency_id
WHERE u.user_id = 1;
```

The screenshot displays the MySQL Workbench interface. The SQL editor contains the following query:

```
SELECT
    u.nombre,
    m.currency_name,
    m.currency_symbol
FROM usuario u
INNER JOIN moneda m
    ON u.currency_id = m.currency_id
WHERE u.user_id = 1;
```

The Result Grid shows the following data:

nombre	currency_name	currency_symbol
Ana Perez	Peso Chileno	CLP

The Output pane shows the execution log, including the final query result:

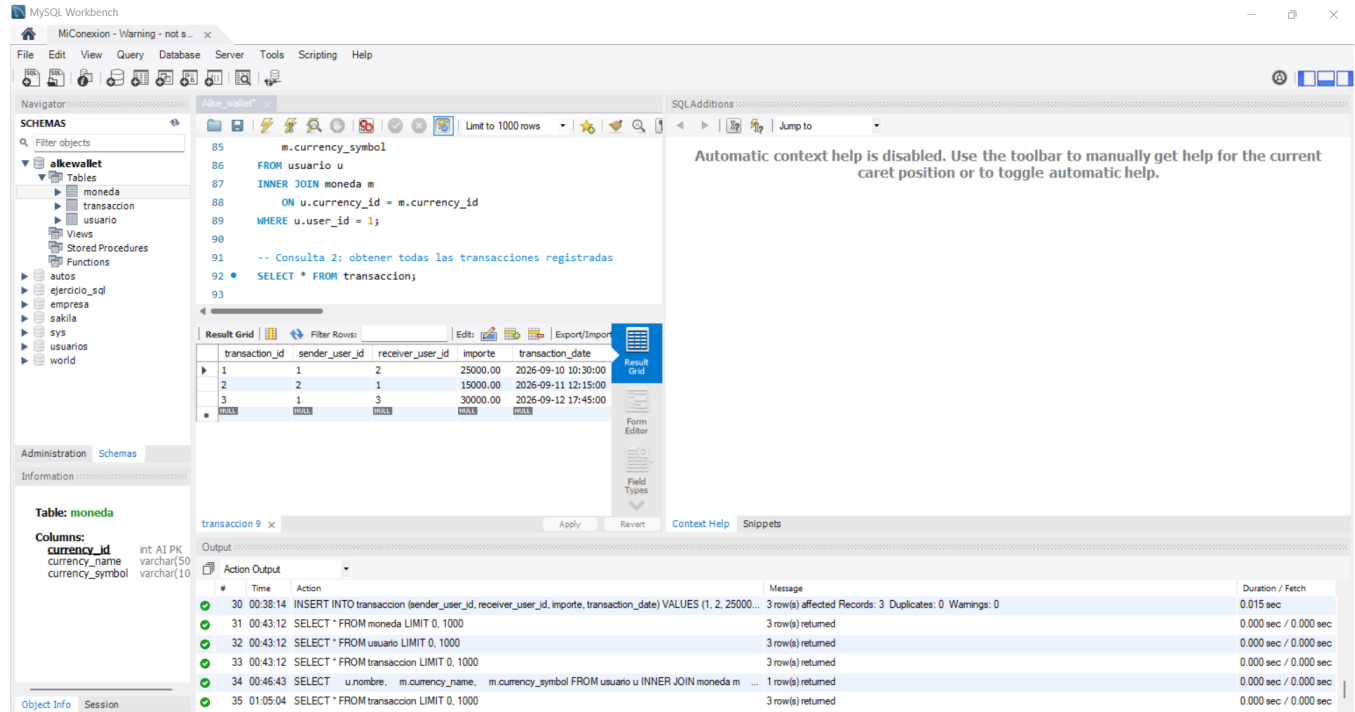
```
34 00:46:43 SELECT u.nombre, m.currency_name, m.currency_symbol FROM usuario u INNER JOIN moneda m ... 1 row(s) returned 0.000 sec / 0.000 sec
```

Figura 4. Resultado: Ana Perez utiliza Peso Chileno (CLP).

7. Consulta 2: todas las transacciones

La segunda consulta recupera todas las filas almacenadas en la tabla transaccion. En el momento de esta ejecución existían tres transacciones.

```
SELECT * FROM transaccion;
```



The screenshot shows the MySQL Workbench interface. The SQL editor contains the query: `SELECT * FROM transaccion;`. The result grid displays the following data:

transaction_id	sender_user_id	receiver_user_id	importe	transaction_date
1	1	2	25000.00	2026-09-10 10:30:00
2	2	1	15000.00	2026-09-11 12:15:00
3	1	3	30000.00	2026-09-12 17:45:00

The output pane shows the execution log with the following messages:

- 30 00:38:14 INSERT INTO transaccion (sender_user_id, receiver_user_id, importe, transaction_date) VALUES (1, 2, 25000.00, '2026-09-10 10:30:00') 3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
- 31 00:43:12 SELECT * FROM moneda LIMIT 0, 1000 3 row(s) returned
- 32 00:43:12 SELECT * FROM usuario LIMIT 0, 1000 3 row(s) returned
- 33 00:43:12 SELECT * FROM transaccion LIMIT 0, 1000 3 row(s) returned
- 34 00:46:43 SELECT u.nombre, m.currency_name, m.currency_symbol FROM usuario u INNER JOIN moneda m ON u.currency_id = m.currency_id WHERE u.user_id = 1; 1 row(s) returned
- 35 01:05:04 SELECT * FROM transaccion LIMIT 0, 1000 3 row(s) returned

Figura 5. Resultado de las tres transacciones registradas inicialmente.

8. Consulta 3: transacciones de un usuario específico

Para obtener las transacciones donde participa un usuario se filtra tanto por `sender_user_id` como por `receiver_user_id`. El operador `OR` permite considerar al usuario como emisor o receptor.

```
SELECT *
FROM transaccion
WHERE sender_user_id = 1
   OR receiver_user_id = 1;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Consulta 3: transacciones de un usuario específico
SELECT *
FROM transaccion
WHERE sender_user_id = 1
   OR receiver_user_id = 1;
```

The query has been executed, and the results are displayed in the Result Grid. The table has 5 columns: `transaction_id`, `sender_user_id`, `receiver_user_id`, `importe`, and `transaction_date`. The results are as follows:

transaction_id	sender_user_id	receiver_user_id	importe	transaction_date
1	1	2	25000.00	2026-09-10 10:30:00
2	2	1	15000.00	2026-09-11 12:15:00
3	1	3	30000.00	2026-09-12 17:45:00

The bottom pane shows the Output window with a list of actions and their execution details:

#	Time	Action	Message	Duration / Fetch
31	00:43:12	SELECT * FROM moneda LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
32	00:43:12	SELECT * FROM usuario LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
33	00:43:12	SELECT * FROM transaccion LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
34	00:46:43	SELECT u.nombre, m.currency_name, m.currency_symbol FROM usuario u INNER JOIN moneda m	1 row(s) returned	0.000 sec / 0.000 sec
35	01:05:04	SELECT * FROM transaccion LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
36	03:32:22	SELECT * FROM transaccion WHERE sender_user_id = 1 OR receiver_user_id = 1 LIMIT 0, 1000	3 row(s) returned	0.015 sec / 0.000 sec

Figura 6. Transacciones en las que participa el usuario con `user_id = 1`.

9. Consulta 4: modificación del correo electrónico

Se utilizó UPDATE para modificar el correo del usuario 1. Después se ejecutó un SELECT de comprobación para verificar el cambio.

```
UPDATE usuario
SET correo = 'ana.perez@email.com'
WHERE user_id = 1;
```

```
SELECT user_id, nombre, correo
FROM usuario
WHERE user_id = 1;
```

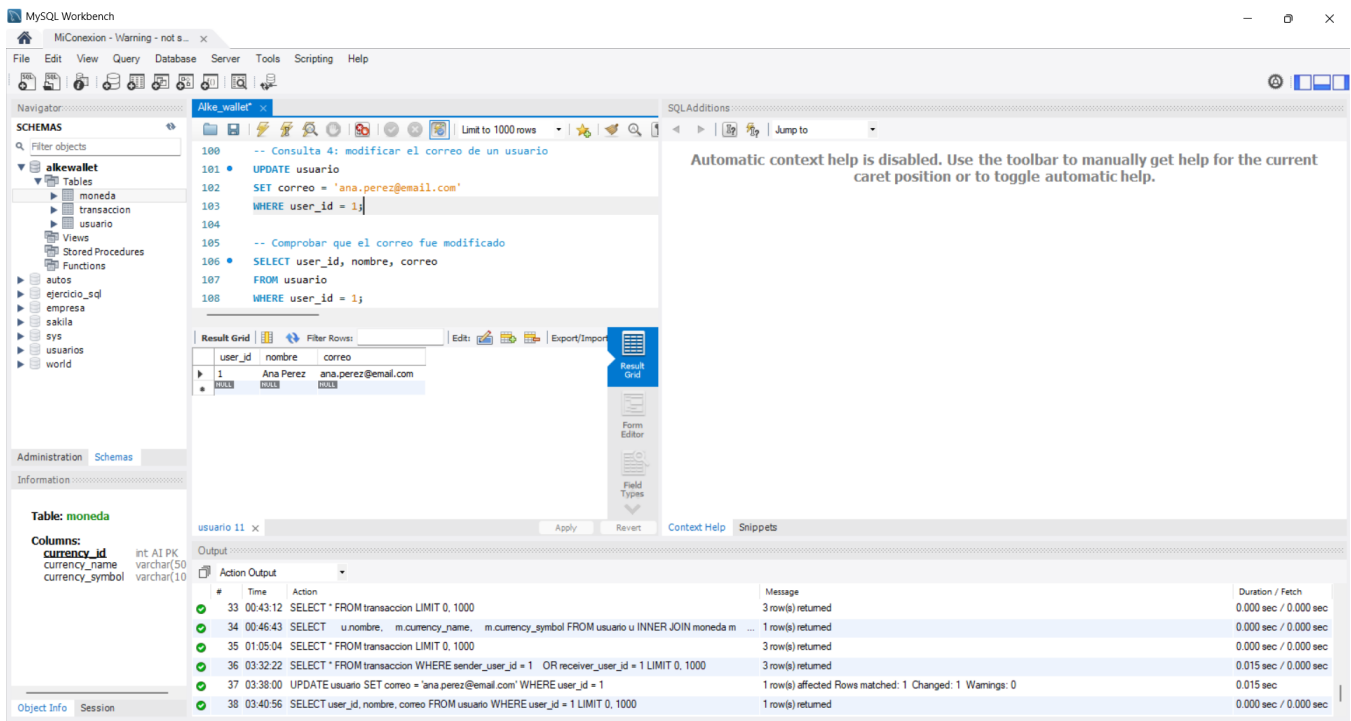


Figura 7. Correo de Ana Perez modificado y comprobado correctamente.

10. Consulta 5: eliminación de una transacción

Se eliminó la fila completa correspondiente a `transaction_id = 3`. El SELECT posterior muestra que la tabla pasó de tres a dos transacciones.

```
DELETE FROM transaccion
WHERE transaction_id = 3;
```

```
SELECT * FROM transaccion;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with 'alkewallet' selected, showing tables 'moneda', 'transaccion', and 'usuario'. The main editor window contains the following SQL script:

```

109 -- Consulta 5: eliminar una transaccion
110
111 • DELETE FROM transaccion
112   WHERE transaction_id = 3;
113
114 -- Comprobar que la transaccion fue eliminada
115 • SELECT * FROM transaccion;
116
117
  
```

The 'Result Grid' shows the result of the SELECT query, displaying two rows of transaction data:

transaction_id	sender_user_id	receiver_user_id	importe	transaction_date
1	1	2	25000.00	2026-09-10 10:30:00
2	2	1	15000.00	2026-09-11 12:15:00

The bottom panel shows the 'Output' tab with a log of database actions:

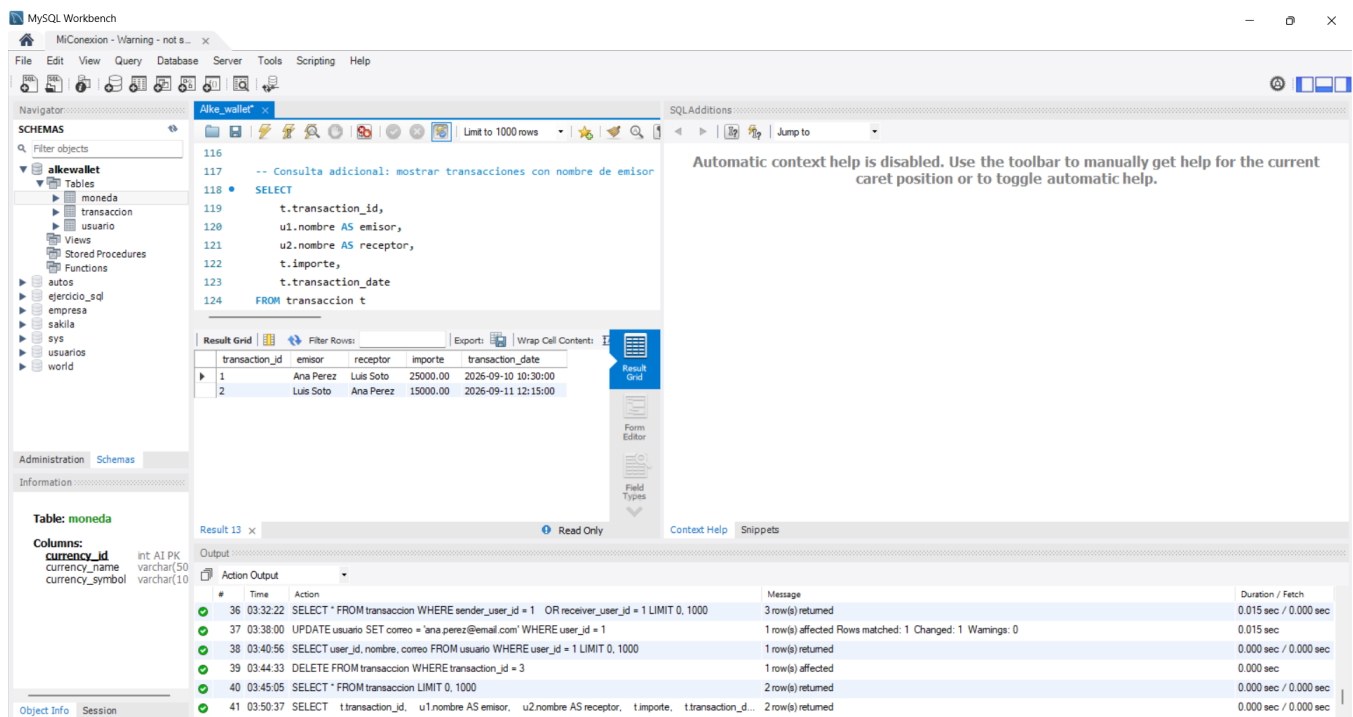
#	Time	Action	Message	Duration / Fetch
35	01:05:04	SELECT * FROM transaccion LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
36	03:32:22	SELECT * FROM transaccion WHERE sender_user_id = 1 OR receiver_user_id = 1 LIMIT 0, 1000	3 row(s) returned	0.015 sec / 0.000 sec
37	03:38:00	UPDATE usuario SET correo = 'ana.perez@email.com' WHERE user_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec
38	03:40:56	SELECT user_id, nombre, correo FROM usuario WHERE user_id = 1 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
39	03:44:33	DELETE FROM transaccion WHERE transaction_id = 3	1 row(s) affected	0.000 sec
40	03:45:05	SELECT * FROM transaccion LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

Figura 8. Eliminación de la transacción 3; quedan dos registros.

11. Consulta adicional: JOIN con nombres de emisor y receptor

Para presentar las transacciones de forma más comprensible se relacionó dos veces la tabla usuario con transaccion: una para obtener el nombre del emisor y otra para obtener el nombre del receptor.

```
SELECT
    t.transaction_id,
    u1.nombre AS emisor,
    u2.nombre AS receptor,
    t.importe,
    t.transaction_date
FROM transaccion t
INNER JOIN usuario u1
    ON t.sender_user_id = u1.user_id
INNER JOIN usuario u2
    ON t.receiver_user_id = u2.user_id;
```



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Alke_wallet' database schema with tables 'moneda', 'transaccion', and 'usuario'. The central SQL editor contains the following query:

```
-- Consulta adicional: mostrar transacciones con nombre de emisor
SELECT
    t.transaction_id,
    u1.nombre AS emisor,
    u2.nombre AS receptor,
    t.importe,
    t.transaction_date
FROM transaccion t
INNER JOIN usuario u1
    ON t.sender_user_id = u1.user_id
INNER JOIN usuario u2
    ON t.receiver_user_id = u2.user_id;
```

The 'Result Grid' shows the following data:

transaction_id	emisor	receptor	importe	transaction_date
1	Ana Perez	Luis Soto	25000.00	2026-09-10 10:30:00
2	Luis Soto	Ana Perez	15000.00	2026-09-11 12:15:00

The bottom panel shows the 'Output' tab with a list of executed queries and their results, including the current query which returned 2 rows.

Figura 9. JOIN exitoso mostrando emisor, receptor, importe y fecha.

12. Consulta adicional: COUNT, SUM y GROUP BY

Esta consulta cuenta cuántas transacciones ha enviado cada usuario y suma el importe enviado. LEFT JOIN permite que también aparezcan usuarios sin transacciones enviadas.

```
SELECT
    u.nombre,
    COUNT(t.transaction_id) AS cantidad_transacciones,
    SUM(t.importe) AS total_enviado
FROM usuario u
LEFT JOIN transaccion t
    ON u.user_id = t.sender_user_id
GROUP BY u.user_id, u.nombre;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Consulta adicional: cantidad de transacciones y total enviado
SELECT
    u.nombre,
    COUNT(t.transaction_id) AS cantidad_transacciones,
    SUM(t.importe) AS total_enviado
FROM usuario u
LEFT JOIN transaccion t
    ON u.user_id = t.sender_user_id
```

The Results grid displays the following data:

nombre	cantidad_transacciones	total_enviado
Ana Perez	1	25000.00
Luis Soto	1	15000.00
Maria Rojas	0	0.00

The bottom panel shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
37	03:38:00	UPDATE usuario SET correo = 'ana.perez@email.com' WHERE user_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec
38	03:40:56	SELECT user_id, nombre, correo FROM usuario WHERE user_id = 1 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
39	03:44:33	DELETE FROM transaccion WHERE transaction_id = 3	1 row(s) affected	0.000 sec
40	03:45:05	SELECT * FROM transaccion LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
41	03:50:37	SELECT t.transaction_id, u1.nombre AS emisor, u2.nombre AS receptor, t.importe, t.transaction_id...	2 row(s) returned	0.000 sec / 0.000 sec
42	03:53:42	SELECT u.nombre, COUNT(t.transaction_id) AS cantidad_transacciones, SUM(t.importe) AS total_enviado...	3 row(s) returned	0.016 sec / 0.000 sec

Figura 10. Agregación por usuario mediante COUNT, SUM y GROUP BY.

13. Transaccionalidad y ROLLBACK

Se simuló una transferencia de \$10.000 desde Ana hacia Luis dentro de START TRANSACTION. Se actualizaron ambos saldos y se agregó una transacción temporal. Finalmente se utilizó ROLLBACK para deshacer todos los cambios de la simulación.

```
START TRANSACTION;
```

```
UPDATE usuario  
SET saldo = saldo - 10000  
WHERE user_id = 1;
```

```
UPDATE usuario  
SET saldo = saldo + 10000  
WHERE user_id = 2;
```

```
INSERT INTO transaccion  
(sender_user_id, receiver_user_id, importe, transaction_date)  
VALUES (1, 2, 10000, NOW());
```

```
SELECT user_id, nombre, saldo  
FROM usuario  
WHERE user_id IN (1, 2);
```

```
SELECT * FROM transaccion;
```

```
ROLLBACK;
```

```
SELECT user_id, nombre, saldo  
FROM usuario  
WHERE user_id IN (1, 2);
```

```
SELECT * FROM transaccion;
```

MySQL Workbench

File Edit View Query Database Server Tools Scripting Help

Navigator: Alke_wallet

SCHMAS

Filter objects

alkewallet

Tables

moneda

transaccion

usuario

Views

Stored Procedures

Functions

autos

ejercicio_sd

empresa

sakila

sys

usuarios

world

Administration Schemas

Information

Table: moneda

Columns:

currency_id int AI PK

currency_name varchar(50)

currency_symbol varchar(10)

SQL Editor

Limit to 1000 rows

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

transaction_id	sender_user_id	receiver_user_id	importe	transaction_date
1	1	2	25000.00	2026-09-10 10:30:00
2	2	1	15000.00	2026-09-11 12:15:00

Form Editor

Field Types

usuario 17 transaction 18

Apply Revert Context Help Snippets

Output

#	Time	Action	Message	Duration / Fetch
46	12:14:57	INSERT INTO transaccion (sender_user_id, receiver_user_id, importe, transaction_date) VALUES (1, 2, 10000...	1 row(s) affected	0.000 sec
47	12:14:57	SELECT user_id, nombre, saldo FROM usuario WHERE user_id IN (1, 2) LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
48	12:14:57	SELECT * FROM transaccion LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
49	12:14:57	ROLLBACK	0 row(s) affected	0.000 sec
50	12:47:40	SELECT user_id, nombre, saldo FROM usuario WHERE user_id IN (1, 2) LIMIT 0, 1000	2 row(s) returned	0.015 sec / 0.000 sec
51	12:47:40	SELECT * FROM transaccion LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Figura 11. Ejecución de la transacción de prueba y ROLLBACK; la tabla vuelve a dos registros.

14. Principios ACID aplicados

Propiedad	Significado	Aplicación en el proyecto
Atomicidad	Una transacción se completa como una unidad.	La transferencia simulada incluye descuento, abono e INSERT dentro del mismo bloque.
Consistencia	Los datos deben conservar reglas válidas.	Las claves primarias, foráneas y NOT NULL ayudan a mantener datos coherentes.
Aislamiento	Las operaciones de una transacción se controlan como un bloque.	START TRANSACTION permite controlar los cambios antes de confirmarlos o revertirlos.
Durabilidad	Los cambios confirmados permanecen almacenados.	En una operación real se utilizaría COMMIT para hacer permanentes los cambios.

Se comprobó ROLLBACK con una transferencia de prueba de \$10.000: los cambios de saldos y el registro temporal se deshicieron. En una prueba posterior, COMMIT confirmó una transferencia de \$5.000, por lo que esos cambios sí permanecieron guardados. Ambas pruebas se realizaron en momentos distintos.

14.1 Integridad de los datos

La integridad se apoya en PRIMARY KEY para identificar cada registro, FOREIGN KEY para impedir relaciones inválidas, NOT NULL para exigir datos necesarios y UNIQUE en el correo para evitar correos repetidos dentro de usuario.

15. Estado final de la base de datos

Al finalizar las cinco consultas obligatorias, la base tenía tres monedas, tres usuarios y dos transacciones: se eliminó la transacción 3 y se modificó el correo de Ana Perez. Posteriormente, como prueba adicional de COMMIT, se confirmó una transferencia de \$5.000 desde Ana hacia Luis y se agregó la transacción 5. Por ello, el estado final verificado muestra tres transacciones (1, 2 y 5).

16. Conclusión

Realizamos la creación de una base de datos relacional desde cero, donde definimos tablas con claves primarias y foráneas, insertamos consultar datos, relacionamos tablas con JOIN y utilizamos funciones como COUNT y SUM. Se utilizó también UPDATE y DELETE, y comprendí que una transacción puede controlarse con START TRANSACTION y ROLLBACK para proteger la consistencia de la información.

El modelo de Alke Wallet permite separar usuarios, monedas y transacciones de manera ordenada, evitando repetir información innecesaria y manteniendo relaciones claras entre los datos.

17. Observación sobre la pauta

La pauta también presenta tareas Plus y recursos de práctica, como índices compuestos, ALTER TABLE, vistas, carga masiva de transacciones y exportación del ERD. Estas actividades se presentan como complementarias u opcionales. El documento se concentra en las sentencias y evidencias efectivamente desarrolladas durante la implementación, junto con el modelo ER, la explicación de normalización y los principios ACID.

18. Prueba adicional: transferencia confirmada con COMMIT

Después de comprobar ROLLBACK, se ejecutó una transferencia de prueba de \$5.000 del usuario 1 (Ana Perez) al usuario 2 (Luis Soto). Se descontó el importe al emisor, se sumó al receptor y se insertó el movimiento en la tabla transaccion. COMMIT confirmó los tres cambios como una sola operación. Esta prueba es adicional a las cinco consultas obligatorias.

```
START TRANSACTION;
UPDATE usuario SET saldo = saldo - 5000 WHERE user_id = 1;
UPDATE usuario SET saldo = saldo + 5000 WHERE user_id = 2;
INSERT INTO transaccion
(sender_user_id, receiver_user_id, importe, transaction_date)
VALUES (1, 2, 5000, NOW());
COMMIT;
SELECT user_id, nombre, saldo FROM usuario WHERE user_id IN (1, 2);
SELECT * FROM transaccion;
```

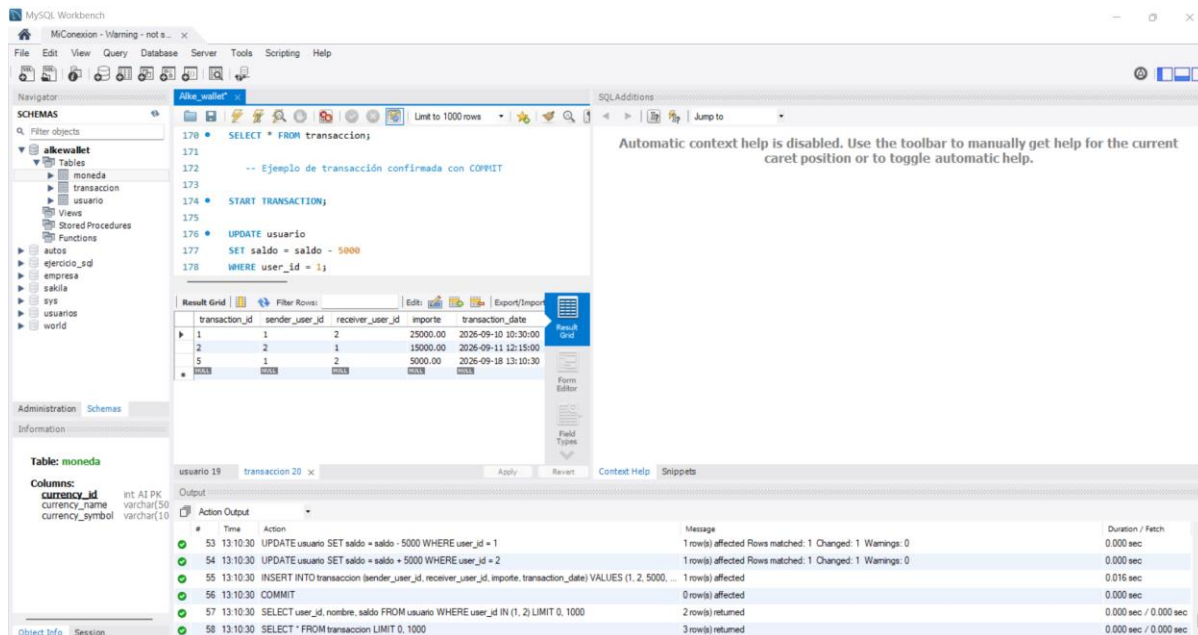


Figura 12. La ejecución de COMMIT confirma una transferencia de \$5.000 y la inserción de una nueva transacción.

En la comprobación posterior se observan las transacciones 1, 2 y 5. El identificador 5 no implica que falten registros por cargar: los identificadores eliminados o usados en pruebas revertidas no necesariamente se reutilizan.

19. Verificación de la estructura de las tablas

Con SHOW TABLES se verificó la existencia de moneda, usuario y transaccion. Con DESCRIBE se revisaron sus campos, tipos de datos y claves. La captura muestra la estructura de transaccion, que incluye la clave primaria transaction_id y las columnas sender_user_id y receiver_user_id utilizadas como claves foráneas.

```
SHOW TABLES;
DESCRIBE moneda;
DESCRIBE usuario;
DESCRIBE transaccion;
```

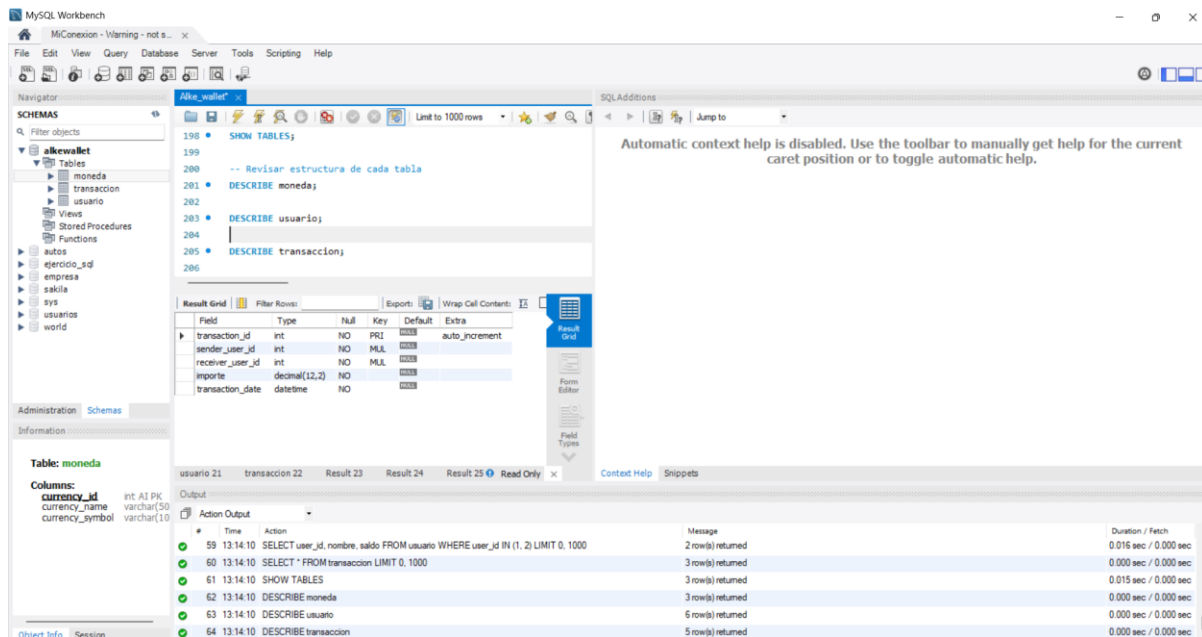


Figura 13. Revisión de los campos y claves de la tabla transaccion.

20. Prueba de integridad referencial

Se intentó insertar una transacción cuyo emisor tenía `user_id = 999`. Como no existe ese usuario en los datos de prueba, MySQL rechazó la inserción con el error 1452 por la restricción de clave foránea. El error es el resultado esperado de esta prueba y demuestra que no se admiten transacciones con emisores inexistentes. Después se ejecutó `ROLLBACK`.

```
START TRANSACTION;
INSERT INTO transaccion
(sender_user_id, receiver_user_id, importe, transaction_date)
VALUES (999, 1, 10000, NOW());
ROLLBACK;
SELECT * FROM transaccion;
```

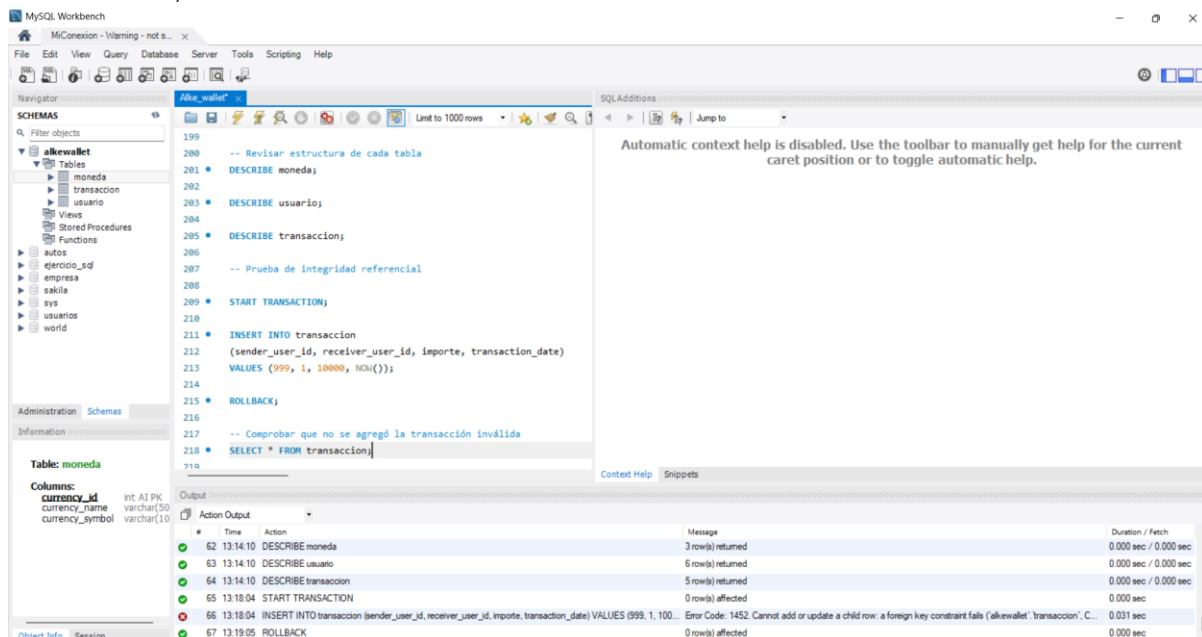


Figura 14. MySQL impide insertar una transacción con un emisor inexistente (error 1452).

La consulta de comprobación posterior conserva las transacciones válidas 1, 2 y 5; la operación inválida no creó una nueva fila.

21. Índice compuesto para consultas de transacciones

Como mejora adicional, se creó un índice compuesto sobre los identificadores del emisor y del receptor. SHOW INDEX permitió comprobar que idx_emisor_receptor aparece asociado a ambas columnas. El índice puede ayudar a ciertas búsquedas que utilizan esas columnas, aunque su beneficio concreto depende de la consulta y de los datos.

```
USE AlkeWallet;
CREATE INDEX idx_emisor_receptor
ON transaccion (sender_user_id, receiver_user_id);
SHOW INDEX FROM transaccion;
```

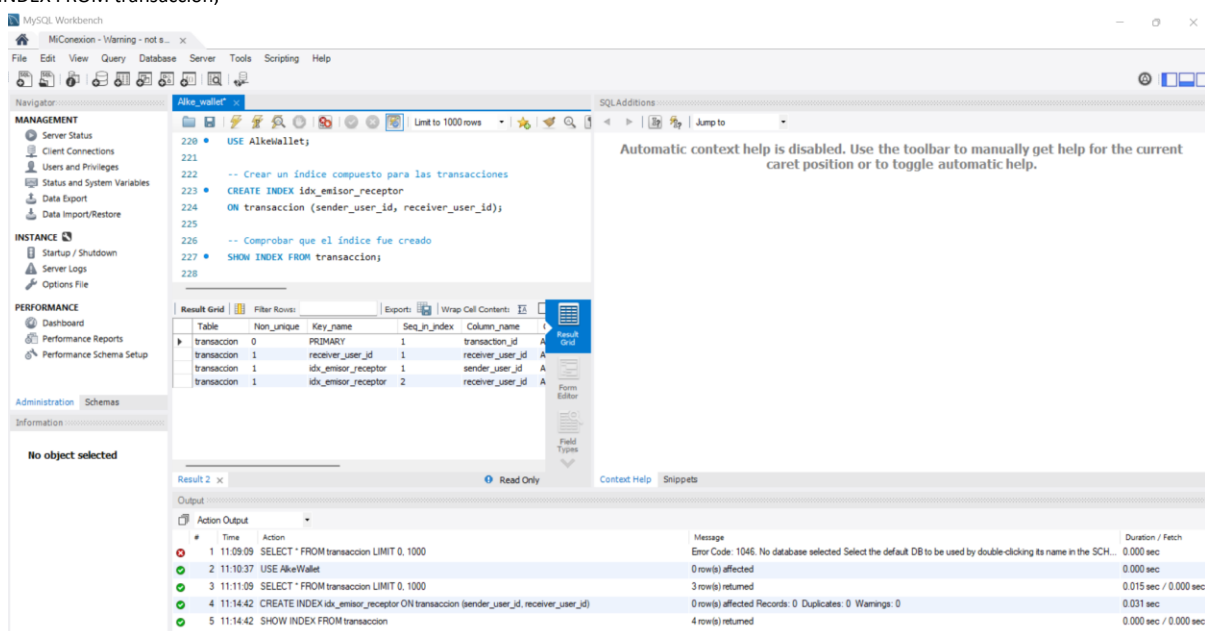


Figura 15. Índice compuesto idx_emisor_receptor creado y verificado.

22. Cierre del proyecto y estado final comprobado

El proyecto incluye el modelo entidad-relación, la creación de las tres tablas y sus relaciones, los INSERT iniciales, las cinco consultas solicitadas en la consigna y las capturas de sus resultados. También se documentan consultas con JOIN y agregaciones, una transferencia revertida con ROLLBACK, otra confirmada con COMMIT, la comprobación de claves foráneas y la creación de un índice compuesto.

Estado final de las pruebas: tres monedas, tres usuarios y tres transacciones válidas (ID 1, 2 y 5). La captura que muestra únicamente las transacciones 1 y 2 corresponde al momento posterior al DELETE y anterior a la prueba con COMMIT; por eso ambas evidencias son correctas dentro de la secuencia de trabajo.